

Assignment 8: Domos — Smart Home IoT Manager

Pending



Objective

Build a **FastAPI-based Smart Home IoT Manager** to register smart devices, manage their states, and maintain logs for every state change.

The assignment should demonstrate:

- Custom Response Headers
- Middleware for Logging
- Enum Validation
- Device State Management
- Query Parameter Filtering
- Device Change Logs

Technical Requirements

- **Framework:** FastAPI
- Device types must use an **Enum**
- Middleware must log:
 - Request endpoint
 - Request execution time
- Add a **custom response header** for request processing time
- Maintain only the **latest 10 state-change logs** for each device
- Device ID must be unique

Device Types

Use an Enum with the following values:

- light
 - thermostat
 - fan
 - camera
 - lock
-

Data Models

Device

- id
- name
- type
- room
- state

DeviceLog

- id
 - device_id
 - old_state
 - new_state
 - timestamp
-

API Routes

POST /devices

Register a new smart device.

Required fields:

- Device ID
- Name
- Device type
- Room
- Initial state

Validation:

- Device ID must be unique.
 - Device type must be a valid Enum value.
-

GET /devices

List devices available in a specific room.

Support filtering using:

room

Example:

`/devices?room=Living Room`

The room filter must be **case-insensitive**.

For example:

living room

Living Room

LIVING ROOM

should return the same matching devices.

PATCH /devices/{id}/state

Update the state of a device.

Example:

on → off

Whenever the state changes, a new `DeviceLog` must be created containing:

- Device ID
- Old state
- New state
- Timestamp

Only the **latest 10 state-change logs** should be maintained for each device.

Return 404 Not Found if the device does not exist.

GET /devices/{id}/logs

Return the latest **10 state changes** for the selected device.

Logs should be returned from newest to oldest.

Return 404 Not Found if the device does not exist.

DELETE /devices/{id}

Unregister a device.

Requirements:

- Delete the selected device.
 - Return 404 Not Found if the device does not exist.
-

Middleware

Create middleware that logs:

- Requested endpoint
- Request execution time

Example:

PATCH /devices/101/state completed in 8.45 ms

Also include the processing time in a **custom response header**.

Example:

X-Process-Time: 8.45 ms

Validation & Business Rules

- Device ID must be unique.
- Device type must be a valid Enum.
- Every state change must generate a log.
- Only the latest 10 logs should be maintained per device.
- Room filtering must be case-insensitive.
- Invalid device IDs must return 404 Not Found.

Bonus

Implement state validation based on device type.

Examples:

Light → on / off

Fan → on / off

Lock → locked / unlocked

Thermostat → numeric temperature value

Camera → on / off

Error Handling

Handle appropriate errors for:

- Device not found
- Duplicate device ID
- Invalid device type
- Invalid state
- Invalid request data

Use appropriate HTTP status codes such as 400, 404, and 422.

API Testing

All APIs should be testable through FastAPI Swagger:

/docs

Test:

- Device registration
 - Duplicate device validation
 - Room filtering
 - State updates
 - State-change logging
 - Latest 10 logs
 - Device deletion
 - Middleware execution time
 - Custom response headers
-

Submission Instructions

Submit the **complete project as ONE ZIP file through the LMS.**

The ZIP file should include:

- Complete source code
- requirements.txt
- README.md
- All files required to run the project

ZIP File Name

StudentName_Domos.zip

Example:

RahulKumar_Domos.zip

Important

- Submit only **one ZIP file**.
 - Do not upload individual files separately.
 - Do not include `venv`, `.venv`, `__pycache__`, or unnecessary folders.
 - Test the application before submission.
 - Ensure the correct ZIP file is uploaded successfully to the LMS.
 - Submit before the deadline mentioned in the LMS.
-

Evaluation Criteria

The assignment will be evaluated based on:

- Correct API functionality
- Device type Enum implementation
- Middleware implementation
- Custom response headers
- Device state update logic
- State-change logging
- Latest 10 log handling
- Case-insensitive room filtering
- Validation and error handling
- Code quality and organization
- Successful project execution
- Proper ZIP submission

Expected Outcome

By completing this assignment, you should be able to build a structured **FastAPI Smart Home IoT Manager using middleware, custom response headers, Enum validation, device state management, filtering, and state-change logging**.

Upload Assignment